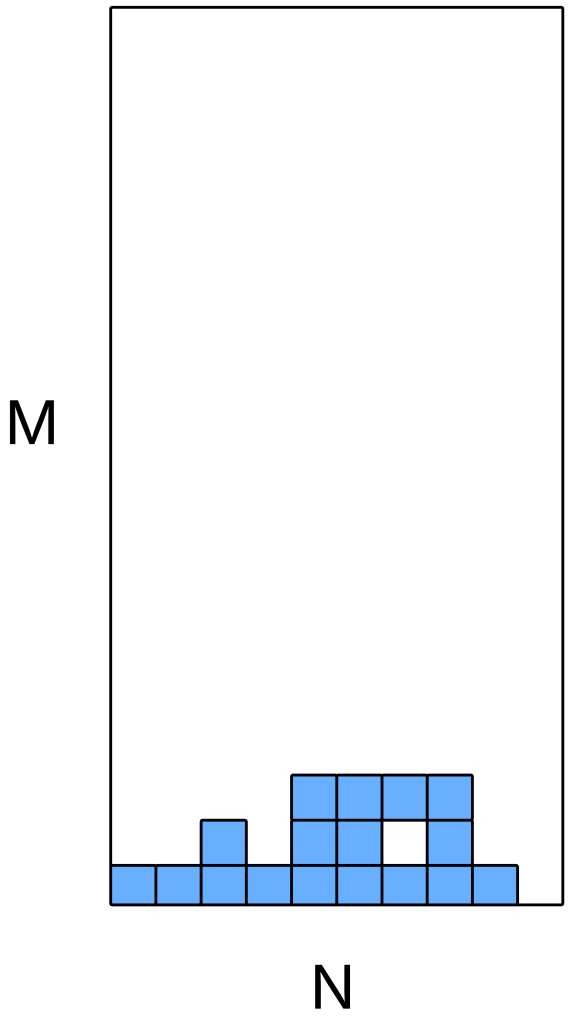
# Tetris




-Heights
-Holes
-Bumpiness
-Total height
-complete lines

Modes availables ← [manual, agent]

Environment ← [arena, tetrominoes]

Current piece ← random([tetrominoes])

Next piece ← random([tetrominoes])

Line rewards ← {0:0, 1:40, 2:100, 3:300, 4:1200}

# Reinforcement Learning agent

Actions ← [rotation, column]

States ← [heights, holes, bumpiness, total height, completed lines]

dim(States) ← 14

Description of states (from the example of the board).

heights ← [1, 1, 2, 1, 3, 3, 3, 3, 1, 0]

holes ← 1, empty cells with at least one filled cell above in the same column

bumpiness ← roughness

$$|h_0 - h_1| + |h_1 - h_2| + ... + |h_(n-1) - h_n|$$

total height ← sum(heights)

complete lines ← int(0) : from the example

# Basic learning algorithm

States ← $[f_0, f_1, ..., f_n]$

Action space ← $[(rotation, column)_0, ..., (rotation, column)_k]$

Q network ←

$$input < -Statevector$$

$$output < -SinglescalarQ(s, a)$$

Policy ← $epsilon - greedy$

Network ← Sequential

$$Linear \rightarrow Relu \rightarrow Linear \rightarrow Relu \rightarrow Linear$$

# Replay buffer (Data Structure)

Store and reuse past experience.

Follows the streaming principle.

- throw data after one usage
- one gradient step between each iteration
- low compute and memory requirements

Parameters.

- Size of replay buffer
- Learning update per iteration

# Training step

buffer sample( batch size ) ← $state_t, action_t, reward_t$

q network prediction ← $qnet(s_t, a_t)$

env ← environment

sequence ← $action, observation, reward$

```
for time_step in sequence T
    action_t draft from A={1, ..., k}
```

epsilon ← stochastic

Sequence at time t ←

$$s_t = x_1, a_1, x_2, ..., a_{t-1}, x_t$$

Large but finite Markov Decision Process (MDP) ←

$$R_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}$$

Q-learning

$$Q^*(s', a') = \max_{\pi} \mathbb{E}[R_t | s_t = s, a_t = a, \pi]$$

$\pi$ ← policy mapping sequence to action or distribution over action

optimal action-value function

## Bellman equation

If the optimal value $Q^*(s', a')$ of the sequence s′ at the next time-step was know for all possible actions a′.

Then the possible strategy is to select the action a′ by maximizing the expected value of:

$$r + \gamma Q^*(s', a')$$

$$Q^*(s, a) = \mathbb{E}_{s' \sim E}[r + \gamma \max_{a'} Q^*(s', a') | s, a]$$

In the practical implementation we dont know $Q^*$ so we approximate it iteratively:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

We stabilize it by using:

- Online network
- Target network

Two ANN to decouple the target from the learning process.

- Online ← $\theta$: update rule every time step t

- Target $\leftarrow \theta^{-}$: copy of online $\theta$

  Computes

  $$r + \gamma \max_{q'} Q(s', a' : \theta^{-})$$

  This network is not trained, every N time steps the weights of the online network are copied over the target network.